Strata — Hướng dẫn Developer

Dành cho kỹ sư triển khai ticket

strata-kb v1.18.0 · 2026-09-19

1. Tài liệu này dành cho ai

Bạn viết code sản phẩm. Ticket đến tay bạn đã được bám vào cơ sở tri thức của tổ chức, kèm trích dẫn ghim vào một phiên bản cụ thể của một tài liệu cụ thể, và bạn hiện thực chúng.

Strata làm hai việc cho bạn.

Nó cho bạn biết bản gốc thực sự nói gì. Không phải "xem sổ tay đi" — mà là đúng section đó, ở đúng phiên bản mà người viết yêu cầu đã đọc, kèm cảnh báo nếu nó đã thay đổi kể từ đó.

Nó publish ra codebase của bạn là gì, để mọi người khác — BA vẽ sơ đồ kiến trúc, nhóm khác, chính bạn trong tương lai — tìm được service, dependency, endpoint và schema của bạn mà không phải đọc repository.

1.1 Repo dev làm gì và không làm gì

Có làm	Không làm
Đọc tri thức đã publish trên hub, gồm cả ticket BA đã publish	Ingest tài liệu từ bên ngoài repo
Publish tri thức về **chính code của nó**	Publish tri thức nghiệp vụ
Chạy quy trình hiện thực gồm năm lệnh	Tự merge hay tự ship bất cứ thứ gì

Ranh giới này là có chủ đích. Repo `child` đưa tài liệu bên ngoài vào; repo `dev` không ingest gì cả và chỉ soạn tri thức về chính nó. Chính điều đó làm cho việc CI của bạn publish liên tục trở nên an toàn.

1.2 Hai tài liệu mà repo của bạn publish

Tài liệu	Nguồn gốc	Sinh lại	Cần review
`<repo_id>-code`	`kb code-ingest` — tất định, không model	khi cần, từ working tree — không bao giờ commit; CI publish ở mỗi lần push lên nhánh mặc định	không
`<repo_id>-svc`	model soạn từ bằng chứng đó, bạn sửa lại	không bao giờ	luôn luôn

`-code` là sự thật máy móc: tên service, công nghệ, dependency, lệnh, endpoint, schema, cấu trúc thư mục. `-svc` là khẳng định của con người: mỗi service thực sự *chịu trách nhiệm* về cái gì.

Chúng nối nhau bằng id section — `svc.<name>` tồn tại ở cả hai — nên một BA vẽ container C4 lấy alias, nhãn và công nghệ từ `-code`, còn phần mô tả từ `-svc`.

2. Thiết lập một lần

```
pip install strata-kb
kb init --kind dev
```

2.1 Cấu hình

```
# .kb/config.yaml
kind: dev
repo_id: payments
hub: https://github.com/acme/kb-hub.git
intake: https://kb-intake.acme.com
```

- **`hub`** là nguồn đọc duy nhất cho `kb query`, `kb get`, `kb resolve` và MCP.
- **`intake`** là cách CI publish. Hãy đề nghị người quản trị hub thêm repository này vào mục `repos:` trong `federation/registry.yaml` của hub — publish sẽ bị từ chối cho tới khi họ làm việc đó.

Nếu KB cần mang schema SQLite, hoặc tag index bổ sung, hãy khai báo ở đây — không phải trong `kb-code.yml` — để mọi lượt refresh cục bộ và CI ingest theo cùng một cách:

```
code_ingest:
 db: [data/app.sqlite]   # phải tường minh
 tags: [payments]
```

2.2 Nối trợ lý của bạn

`.mcp.json` (Claude Code) và `.cursor/mcp.json` (Cursor) đã được scaffold sẵn, nối tới các công cụ MCP của hub — nhưng qua placeholder, `${STRATA_KB_HUB_URL}` và `${STRATA_KB_HTTP_TOKEN}`, mà chưa ai điền giá trị.

Chạy `kb mcp-setup` (trong trợ lý: `/kb-mcp-setup`). Lệnh này hỏi URL HTTP cơ sở và token của hub, ghi cả hai vào `.env`, đảm bảo git bỏ qua file đó, rồi xác minh chúng với hub, để một URL sai và một token bị từ chối trả về hai lỗi khác nhau:

Biến	Ví dụ
`STRATA_KB_HUB_URL`	`http://kb-hub.example.com:8321`
`STRATA_KB_HTTP_TOKEN`	bearer token của hub

Chạy lại một lần nữa (không kèm gì) sẽ xác minh lại mà không cần nhập lại token. Nên dùng prompt ẩn hoặc biến môi trường `STRATA_KB_HTTP_TOKEN` thay vì cờ `--token` — cờ đó để lại token trong lịch sử shell của bạn.

`.mcp.json` và `.cursor/mcp.json` đọc hai biến đó từ môi trường tiến trình, không đọc trực tiếp từ `.env`, nên hãy nạp `.env` vào shell của bạn (`set -a; source .env; set +a`, hoặc dùng direnv) rồi khởi động lại trợ lý — MCP chỉ đọc môi trường lúc khởi động.

2.3 Bật cổng PR

`kb init` ghi ra `.github/workflows/kb-pr-lint.yml` nhưng không bật được branch protection thay bạn. Hãy thêm **`pr-lint` vào danh sách check bắt buộc** của nhánh.

> Hãy quyết định điều này một cách có ý thức. Khác với cổng của repo BA, cổng này không bao giờ tự bỏ qua. Một khi đã bắt buộc, nó chặn **mọi** pull request thiếu các mục bắt buộc — kể cả pull request của bot như nâng cấp dependency

> hay revert.

2.4 Hỏi về chính sách auto-merge của hub

Pull request `-code` trên hub có thể được tự động merge; pull request `-svc` thì tuyệt đối không. Hãy hỏi người quản trị hub cấu hình ra sao, và chắc chắn một điều:

> `kb ci-publish` mở **một** pull request trên hub cho mỗi lần push, không phải mỗi tài liệu một cái. Intake gắn nhánh publish theo id repository và dùng lại nhánh đó trong khi PR còn mở — nên một lần push mang theo `-code` vừa sinh lại cùng với một sửa đổi `-svc` đã merge sẽ đưa **cả hai** vào cùng PR đó.
>
> Vì vậy quy tắc auto-merge phải được **giới hạn theo đường dẫn** `.kb/<repo_id>-code/**`, không bao giờ là quy tắc cho cả PR hay cả repo. Một quy tắc kích hoạt theo kiểu "đây là lượt publish `-code` của repo dev" sẽ âm thầm vô hiệu hoá chính cổng review `-svc` mà việc tách hai tài liệu sinh ra để bảo vệ.

Quy tắc auto-merge này cũng là thứ giữ cho hub không bị tụt lại so với main: thiếu nó, mỗi lần push sẽ dồn vào một PR hub đang chờ mà ai đó phải merge tay. `kb doctor` báo khi `-code` trên hub bị tụt lại.

---

3. Đưa một dự án đang chạy vào hệ thống

Repository của bạn đã có các service đang làm việc thật. Áp dụng Strata không nên có nghĩa là phải viết tài liệu hệ thống từ trang giấy trắng, nên có một bước seed chạy một lần.

```
/dev-code-seed
```

Bảy bước:

#	Bước	Ai làm
1	**Preflight** — xác nhận đã có `hub:`, `repo_id:` và `intake:`, và repo đã được cho phép trên hub; cảnh báo nếu cây làm việc chưa sạch	agent
2	**Extract** — `kb code-ingest --scaffold-svc` sinh lại `-code` và tạo khung `svc.<name>` ở trạng thái `pending` cho mỗi service phát hiện được, kèm bằng chứng code tất định	agent
3	**Draft** — `kb summarize <repo_id>-svc` viết đoạn mô tả trách nhiệm cho từng section từ bằng chứng đó	agent
4	**Review** — rà từng section đã soạn, đối chiếu với bằng chứng, và sửa lại	**bạn**
5	**Approve** — `kb approve <repo_id>-svc` chuyển các section đã sửa sang `reviewed`	bạn
6	**Flows** (tuỳ chọn) — thêm section `flow.<name>` cho các luồng nghiệp vụ đi qua nhiều service	bạn
7	**Validate và publish** — `kb build` không kèm `--allow-pending`, rồi `kb publish --pr`	bạn

3.1 Bước 4 mới là phần việc thật

> **Bản nháp của model là bản nháp, không phải sự thật.** Nó có thể gọi sai luồng, bỏ sót một trách nhiệm, hoặc thổi phồng một trách nhiệm. Duyệt một section mà bạn chưa đọc là phá hỏng toàn bộ ý nghĩa của cái cổng này.

Hãy đọc phần tóm tắt của từng section `svc.*` bên cạnh bằng chứng code của nó và sửa lại. Ở chỗ nào trách nhiệm chạm tới một tiêu chuẩn nghiệp vụ, hãy trích dẫn `doc-id §section` từ hub thay vì diễn đạt lại quy tắc bằng lời của bạn.

Hãy dự trù 10–15 phút cho mỗi service. Đó là chi phí thật, và là chi phí một lần. Sau khi seed xong, `-code` tự sinh lại còn `-svc` tích luỹ dần từng vài dòng.

Với bước 6, cứ bỏ qua thoải mái. Thiếu một luồng vẫn tốt hơn một luồng đoán mò.

3.2 Vì sao lần publish đầu phải `--pr` thủ công

Bước seed diễn ra *trước khi* có bất cứ thứ gì merge vào nhánh mặc định, nên không có lần push nào để `kb-code.yml` phản ứng.

Hãy dùng `kb publish --pr`. Lệnh `kb publish` thường sẽ gắn tag vào commit rồi hỏi dò intake tới mười phút, chờ một lượt CI kích hoạt bằng tag mà repository này không hề có, rồi báo lỗi gây hiểu nhầm.

Pull request đó trên hub được một BA hoặc kiến trúc sư review, như mọi nội dung khác trên hub. `-svc` là tri thức do người biên soạn; nó publish qua review, không bao giờ qua auto-merge.

---

4. Hiện thực một ticket

Năm lệnh, được dựng sẵn cho Claude Code, GitHub Copilot và Cursor.

```
/dev-implement-ticket <ticket>     intake · check · resolve · ground · placeholders
       │
       ├─▶ dev-design       ── A1: reviewer độc lập ── GATE 1: bạn duyệt thiết kế
       ├─▶ dev-plan         ── A2: reviewer độc lập ── GATE 2: bạn duyệt kế hoạch
       ├─▶ dev-execute         A3 mỗi task · A4 cuối nhánh  (lặp lại được, tiếp tục được)
       └─▶ dev-handover     ── A5: merge-risk review ── GATE 3: bạn mở PR
                               GATE 4: bạn merge
```

`/dev-implement-ticket` cũng là bộ điều phối: với một ticket mới, nó chạy năm bước đầu rồi gọi các pha theo thứ tự, tự phát hiện ticket đã đi tới đâu nên không bao giờ làm lại phần đã xong.

4.1 Chỉ cần id ticket là đủ

Chỉ đưa cho trợ lý một id ticket (`M-<slug>-US<n>`, hoặc một tên ticket trơn). Intake lấy ticket **từ hub trước**: BA publish mọi ticket đã sẵn sàng trong tài liệu `<repo_id>-plan`, nên hub là nguồn sự thật.

```
kb ticket fetch M-checkout-US3          # hoặc <repo>:<id> nếu nhiều repo cùng có
```

- Lệnh tìm repo federation duy nhất có tài liệu `*-plan` chứa section đó. Nếu không có, nó thoát `1`: ticket chưa được publish, hãy nhờ BA `kb publish`, hoặc đưa đường dẫn hay dán nội dung. Nếu có nhiều repo, nó thoát `1` và nêu tên chúng; hãy viết id dạng `<repo>:<id>`.

- Nó ghi `docs/impl/<id>/ticket.md` (`--out` để đổi) gồm tiêu đề, một dòng nguồn gốc và nội dung ticket: `> Fetched from hub: <repo>:<doc> §<id> @ <hub rev> — do not edit; amend in the BA repo.`
- Nó **ghi đè** file ở mỗi lần fetch. Bản sao này suy ra được từ hub, nên `docs/impl/.gitignore` bỏ qua `*/ticket.md`: không bao giờ commit, không bao giờ sửa. Nếu ticket sai, BA sửa đổi nó trong repo BA.
- Id không phải một tên trơn (đường dẫn, `..`) bị từ chối.

Khi không có CLI, trợ lý đọc section qua MCP (`kb_get_section`, `<doc> §<id>` ở mức `l3`) và lưu theo cùng cách. Một đường dẫn hay nội dung ticket được dán vẫn dùng được, như phương án dự phòng khi hub không có bản sao hoặc không kết nối được.

Bản trên hub **không có** `## Definition of Ready`: có mặt trên hub nghĩa là BA đã tick mọi ô, nên không bước nào được dừng vì thiếu mục đó.

`## Sequencing` của mission cha được đọc theo cùng cách khi không có file mission cục bộ (`kb get <doc> <M-id> --level l3`), nên bạn thấy được ticket bị chặn bởi gì mà không cần clone repo BA.

Kiểm tra grounding

Ngay sau Intake, luồng chạy `kb ticket check docs/impl/<id>/ticket.md` (chính là `<ticket-file>`; bỏ qua kèm một note khi ticket không có `## Technical grounding`). Lệnh đọc `-code` trên hub và, với `[NEW: D<n>]`, đọc mission cha từ plan trên hub khi không có `missions/` cục bộ. Revision ở `Grounded on:` đổi theo mỗi commit bạn merge, nên chỉ lệch revision thì không bao giờ fail:

Output	Ý nghĩa	Việc cần làm
PASS, `[note] grounded @<old>, still valid at @<new> — cited sections unchanged`	Không thứ gì ticket trích bị đổi	Làm tiếp
PASS, `[warn] … §<id> changed; cited names still hold, review the diff:`	Một section được trích có đổi, mọi tên được trích vẫn còn	Đọc diff rồi quyết; làm tiếp trừ khi nó đổi ý nghĩa ticket
PASS, `[warn] some cited facts could not be verified …`	Một bước check bị bỏ qua (ví dụ giới hạn của tree)	Tự kiểm tra những fact đó
FAIL, `… re-run /sa-ticket-ground`	Một id, cột, route, lệnh, file hay compose fact được trích đã mất	Dừng: blocker cho BA, SA sẽ ground lại ticket
FAIL, kèm `[note] checked against the local -code, not the hub's copy …`	Hub không với tới được hoặc không có `-code`; branch của bạn có thể là nguyên nhân	Chạy lại khi hub với tới được và có `-code`
FAIL không có dòng re-ground	Một open decision hoặc `[NEW: D<k>]` chưa quyết	Báo BA và dừng

Trạng thái của ticket nghĩa là gì với bạn

Menu **Plans** của hub (`/ui/plans`) và `kb mission next --from-hub [--repo <ba-repo>]` hiển thị mọi story của một mission đã publish là **done** (id của nó nằm trong một dòng `hist.*` của `-svc` của bạn, do `kb svc note` ghi lúc handover), **ready** (ticket của nó đã publish), **not ready** (chưa có ticket đã publish) hoặc **blocked** (kèm lý do). Hãy chọn story **ready**.

Ở phía BA, `kb mission next` còn có thể báo một ticket là `draft` với lý do `parent mission rev <M> affects this story`: mission đã được sửa sau khi ticket được viết, và BA chưa soạn lại nó. Ticket như vậy không được publish, nên bạn thấy story là *not ready* và `kb ticket fetch` không tìm thấy gì. Hãy đợi BA; đừng hiện thực từ một bản cũ.

4.2 Chọn điểm vào

Tình huống	Chạy
Ticket mới, chưa bắt đầu gì	`/dev-implement-ticket <ticket>`
Ticket nhỏ, thay đổi đã quá rõ	`/dev-implement-ticket <ticket>` — luồng tự thu gọn; thiết kế vẫn được ghi ra, chỉ là bản rút gọn
Thiết kế đã duyệt, chưa có kế hoạch	`/dev-plan <id>`
Kế hoạch đã duyệt, hoặc đang hiện thực dở	`/dev-execute <id>`
Code đã tự viết, cần phần mô tả PR	`/dev-handover <id>`
Quên mất ticket đang ở đâu	`/dev-implement-ticket <id>`
Chỉ muốn kiểm tra một trích dẫn	`kb resolve <file>`

Mọi điểm vào đều kiểm tra lại độ mới của trích dẫn trước tiên. Hub có thể đã publish kể từ phiên làm việc trước, nên một tham chiếu hôm qua còn `ok` thì hôm nay có thể đã `stale`. Chỉ kiểm tra lúc handover thì quá muộn.

4.3 Công việc nằm ở đâu

File	Do ai ghi	Mục đích
`docs/impl/<id>/design.md`	`dev-design`	bản thiết kế
`docs/impl/<id>/plan.md`	`dev-plan`	mỗi tiêu chí chấp nhận một task, dạng checkbox `- [ ]`, mỗi task một dòng `Depends on:`
`docs/impl/<id>/ticket.md`	`kb ticket fetch`	ticket lấy từ hub; bị gitignore, bị ghi đè ở mỗi lần fetch, không bao giờ sửa
`docs/impl/<id>/context.md`	`dev-implement-ticket`	cache ngữ cảnh đã phân giải; bị gitignore, sinh lại khi cần
`docs/impl/<id>/review/`	`dev-execute`, `dev-handover`	diff và báo cáo reviewer của từng task, diff cả nhánh, báo cáo rủi ro merge; bị gitignore, suy ra được từ git và các bảng review record dưới đây

`kb init --kind dev` và `kb impl tidy` tự dời layout phẳng cũ (`docs/impl/<id>-plan.md`, …) vào các folder này.

File thiết kế và kế hoạch mỗi file đều có thêm bảng `## Review record` khi các vòng review chạy — đây là lịch sử được commit của mọi vòng, không phải một file riêng.

Trạng thái được suy ra từ hai file đầu, nhánh git hiện tại, và việc PR đã mở hay chưa — **không bao giờ từ lịch sử hội thoại**. Bất kỳ pha nào cũng tiếp tục được từ đầu trong một phiên làm việc hoàn toàn mới.

`kb init` không bao giờ sửa nội dung `docs/impl/` của bạn: nó thêm `.gitkeep` và một `.gitignore` để cache ngữ cảnh, ticket đã fetch và các artefact review không bao giờ lọt vào pull request, và dời layout phẳng cũ vào `<id>/` — không bao giờ ghi đè. Hãy commit các lần đổi tên.

Wave và lane. Mỗi task trong kế hoạch có một dòng `Depends on:` ngay dưới tiêu đề `### Task <n>:` — `none`, hoặc các task mà nó dùng Interface hay dùng chung file. `dev-plan` suy ra dòng này; reviewer A2 kiểm tra nó. `kb plan waves docs/impl/<id>/plan.md` biến các dòng đó thành các wave (wave n = các task có mọi dependency nằm ở wave trước) và fail khi hai task dùng chung một file mà không có dependency giữa chúng, khi có vòng lặp, hoặc khi thiếu dòng — kế hoạch như vậy không có thứ tự song song an toàn. Kế hoạch mà không task nào có dòng

này là kế hoạch có từ trước khi có wave, và chạy tuần tự.

`dev-execute` chạy lệnh này sau khi tách nhánh. Wave chỉ có một task chạy như cũ; wave có nhiều task thì mỗi task chạy trong một lane git worktree riêng cắt từ nhánh ticket, tối đa 3 lane cùng lúc, merge lại theo thứ tự task bằng `--no-ff`, toàn bộ test suite chạy một lần mỗi wave. Merge conflict nghĩa là kế hoạch sai, và được trả về `dev-plan`.

4.4 Bốn cổng người, năm vòng review agent

Không gì trong quy trình này merge hay ship mà không có con người.

1. **Thiết kế được duyệt** — bạn ký duyệt trước khi viết bất kỳ kế hoạch nào.
2. **Kế hoạch được duyệt** — bạn ký duyệt trước khi viết bất kỳ dòng code nào.
3. **PR được mở** — agent viết nội dung; bạn mở PR.
4. **Merge** — bạn review và merge.

Agent không tự làm hai việc cuối.

Trước mỗi cổng trong ba cổng đầu, một agent khác đã đọc lại công việc trong một ngữ cảnh riêng — không bao giờ là chính agent đã làm ra nó:

	Ở đâu	Nhìn cái gì
A1	`dev-design`, trước GATE 1	thiết kế có khớp ticket không
A2	`dev-plan`, trước GATE 2	kế hoạch có test-first, có chạy được không
A3	`dev-execute`, mỗi task	diff của task này có đúng spec, code có tốt không
A4	`dev-execute`, sau task cuối	nhánh hoàn chỉnh có làm tròn ticket không
A5	`dev-handover`, trước GATE 3	merge vào nhánh chính có an toàn không

Tiêu chí nằm ở `docs/pr-review-rubric.md`, ghi đè bằng `docs/pr-review-rubric.local.md`. Mỗi vòng tối đa 3 lượt fix/review. Một vòng chỉ tính là sạch khi không còn BLOCKER và không còn SUGGESTED nào bỏ ngỏ — NOTE và NITS chỉ được ghi lại, không bị bắt sửa — và một BLOCKER hoặc SUGGESTED còn đứng sau lượt 3 sẽ dừng luồng và chuyển cho bạn.

Ở nơi runtime của trợ lý không dispatch được subagent — mặc định với wrapper Copilot và Cursor — vòng review vẫn chạy, nhưng chỉ là một lượt đọc đúng những gì được giao, và đó là một **báo cáo, không phải một cổng**: pha đó dừng lại tại chỗ và chuyển kết quả cho bạn, thay vì tự tiến bước dựa trên lời tự nhận của mình.

---

5. Cái gì được máy bảo đảm, cái gì chỉ là lời nhắc

Hãy tỉnh táo về sự khác nhau này.

5.1 Máy bảo đảm

Quy tắc	Được bảo đảm bởi
PR mang theo bằng chứng của nó	`kb pr lint` trong `kb-pr-lint.yml` ở mọi pull request
Tri thức chưa review không lên được hub	`kb build` thoát 1 khi còn section `-svc` ở trạng thái `pending`
Cache ngữ cảnh do CLI sở hữu	`kb resolve --status-only --cache` từ chối cache có version, tập tham chiếu hoặc khối đã phân giải khác với ticket
Merge-risk review đã chạy và không còn BLOCKER	`kb pr lint` — mục `## Testing` phải có dòng `Blocking: No`

Mô tả PR ngắn gọn: `## Ticket` (ticket yêu cầu gì), `## Changes` (PR đã làm gì), `## Testing` (dòng AC→test, lệnh test đã dán kèm các dòng tổng kết, và `Blocking: No` của A5), cộng `## Notes` tùy chọn (ngoài phạm vi, chưa xử lý, phát hiện). `kb pr lint` fail khi thiếu Ticket, Changes hoặc Testing, hoặc mục đó còn nguyên dòng chú thích mẫu, khi Testing không có output đã dán hay không có khối nào chứa lệnh test của kế hoạch, hoặc khi dòng `Blocking:` thiếu hay ghi `Yes`.

5.2 Chỉ là lời nhắc trong prompt — không có gì đo lường

Đây là những quy tắc mà các skill nhắc đi nhắc lại. Không có gì trong `kb` bảo đảm hay đo lường chúng. Phép kiểm tra duy nhất là con người ở cổng — tức là bạn.

- **TDD.** Không viết code sản phẩm khi chưa thấy một test fail trước, ở mọi bước của `dev-execute`. Các loại được miễn được nêu tên trong `docs/tdd-exemptions.md` và khai báo trong kế hoạch.
- **Verify phải có bằng chứng.** Không tuyên bố hoàn thành khi chưa có output lệnh thật. `kb pr lint` kiểm tra nội dung PR, không kiểm tra phiên làm việc.
- **Giá trị đã ghim, nguyên văn.** Mọi giá trị suy ra từ tiêu chuẩn phải lấy từ section đã phân giải ở phiên bản hub đã ghim, kèm chú thích trích dẫn.
- **Ticket là chỉ đọc.** Phát hiện được chuyển ngược về BA; agent không bao giờ sửa ticket.
- **Cổng 1 và cổng 2.** Thiết kế và kế hoạch phải được duyệt trước khi sang pha sau. Dòng `status:` ghi lại điều đó; con người mới là người duyệt.
- **`OPEN(BA)`.** Một tiêu chí chấp nhận mơ hồ phải được đưa lên hỏi, không bao giờ được tự diễn giải lại.
- **A1–A4.** Bốn vòng review agent trước GATE 3 là kỷ luật trong prompt. Không có gì trong `kb` đo được rằng chúng đã chạy. Chỉ A5 có răng: `kb pr lint` đọc dòng `Blocking:` trong mục `## Testing` của PR. Một agent bỏ qua A1–A4 rồi viết một dòng `Blocking: No` trung thực vẫn qua cổng — cổng nằm ở chỗ merge thật sự xảy ra, không phải ở mọi bước trước đó.

---

6. Làm việc với trích dẫn

```
kb resolve tickets/ABC-123.md    # ok | stale | broken, theo từng tham chiếu
kb diff hr-handbook --against <rev>
kb get hr-handbook 4.12 --level l3
```

Trạng thái	Nghĩa là gì	Làm gì
`ok`	Nội dung đã ghim vẫn khớp với bản đang publish	Cứ hiện thực theo đó
`stale`	Nguồn bị sửa đổi sau khi ticket được viết	Chạy `kb diff`, rồi hỏi BA xem tiêu chí chấp nhận còn đúng không. **Đừng âm thầm hiện thực theo bản mới.**
`broken`	Tham chiếu không còn phân giải được	Báo lên. Ticket không hiện thực được như đang viết.

Khi một giá trị trong code của bạn đến từ một tiêu chuẩn — một định dạng, một enum, một ngưỡng, một độ dài trường — hãy lấy nó từ section đã phân giải ở phiên bản đã ghim và để lại một chú thích trích dẫn ngay cạnh. Sáu tháng sau, chú thích đó là thứ duy nhất nối hằng số ấy với lý do tồn tại của nó.

---

7. `kb code-ingest` trích xuất những gì

```
kb code-ingest                          # mặc định thường là đúng
kb code-ingest --db data/app.sqlite     # nêu tên database một cách tường minh
kb code-ingest --json                   # báo cáo dạng máy đọc
```

Bảy extractor, tổ chức theo **loại artefact chứ không theo ngôn ngữ lập trình**, vì Strata được áp dụng trên nhiều dòng dự án.

Extractor	Tiền tố section	Đọc
`services`	`svc.<name>`	file compose, Dockerfile, manifest k8s, file solution và workspace
`deps`	`dep.<ecosystem>`	manifest của Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift, Dart
`commands`	`cmd.<purpose>`	script trong package, Makefile, tox, shell script, bước run trong CI
`tree`	`struct.tree`	các file Git theo dõi, nên virtualenv bị ignore không bao giờ xuất hiện
`schema`	`db.<table>`	migration SQL, Prisma, Alembic, EF, và file SQLite được nêu tên tường minh
`integrations`	`int.<name>`	**chỉ tên khoá** biến môi trường
`api`	`api.<tag>`	hợp đồng OpenAPI và Swagger nằm trong repo

`tree` thì lúc nào cũng tìm ra thứ gì đó, nên quy tắc không-phát-hiện-gì là: `kb code-ingest` thoát `1` khi **không extractor nào ngoài `tree`** tìm được gì. Một tài liệu chỉ chứa danh sách thư mục là cấu hình sai, không phải tri thức.

7.1 Hai thuộc tính an toàn

Đây là thuộc tính thiết kế, không phải tiện ích.

1. **Chỉ lấy khoá, không bao giờ lấy giá trị.** File môi trường duy nhất mà extractor integrations mở là `.env.example`, `.sample` hoặc `.template` — không bao giờ là `.env` thật — và biểu thức của nó không có nhóm bắt nào quanh phần giá trị, nên không có gì để rò rỉ dù dòng khớp chứa gì. Khối `environment:` trong compose **có** được đọc và có chứa giá trị thật, nhưng chỉ tên khoá được giữ lại.
2. **Đầu vào database phải tường minh.** Schema SQLite chỉ được đọc từ đường dẫn bạn nêu trên dòng lệnh. Một database nháp hay fixture trong repository không bao giờ vô tình trở thành tri thức đã publish của công ty.

7.2 Giới hạn của bộ phân tích, nói rõ từ đầu

Đây là giới hạn, không phải lỗi.

- `schema` nhận diện `CREATE TABLE` và `ALTER TABLE … ADD COLUMN` theo thứ tự tên file đã sắp xếp. Nó không hiện thực một phương ngữ SQL.
- Một `CREATE TABLE` cho tên đã được tạo ở một thư mục migration *khác* sẽ bị bỏ kèm cảnh báo nêu tên cả hai file, và `ADD COLUMN` chỉ áp dụng trong thư mục của `CREATE TABLE` tương ứng. Hai schema độc lập trùng tên bảng không bao giờ bị gộp thành một thứ không tồn tại ở đâu cả.
- `RENAME`, `DROP`, `ADD CONSTRAINT` và `ADD INDEX` được cảnh báo là ngoài phạm vi.

- Tên bảng không khớp dạng định danh nhận diện được sẽ bị bỏ qua **kèm cảnh báo nêu tên file**. Bộ đọc đếm số từ khoá `CREATE TABLE` và so với số nó nhận diện được, nên việc bỏ qua không bao giờ im lặng.
- Migration của EF Core chỉ được nhận diện theo tên bảng.
- `build.gradle` được khớp bằng regex, không phân tích như một DSL.
- Maven `<plugins>`, `setup.cfg` `extras_require`, `pnpm-workspace.yaml` và `healthcheck:` trong compose không được đọc.

7.3 Đích đến mà lệnh này không tạo ra sẽ bị từ chối

Nếu `.kb/<doc_id>/` đã có Markdown hoặc manifest, mà manifest đó thiếu, không đọc được, không mang tiêu đề `… — code knowledge`, hoặc liệt kê một id section nằm ngoài bảy tiền tố được sinh ra, thì `kb code-ingest` thoát 1 và không ghi gì cả. Không có cờ ghi đè — hãy chuyển tài liệu đi hoặc chọn `--doc-id` khác.

7.4 Tính tất định và lưu ý về cây làm việc chưa sạch

Chạy lại trên cùng một commit, cùng một nền tảng sẽ cho file giống nhau từng byte — chính điều đó làm cho lượt publish trong CI thực sự là no-op khi cây file không đổi.

Hai giới hạn được nói thẳng: một vài phép khớp glob chuẩn hoá chữ hoa/thường theo hệ điều hành, nên đảm bảo này là theo từng nền tảng (CI luôn chạy Linux nên thứ được publish vẫn ổn định); và `revision` trong manifest suy ra từ **commit HEAD** trong khi cây làm việc của bạn có thể đang có thay đổi chưa commit. `kb code-ingest` phát hiện điều đó và in cảnh báo `dirty_tree`.

`-code` cục bộ là thứ CI sẽ sinh ra nếu các file đang được track của bạn được commit ngay bây giờ; `kb doctor` cảnh báo khi strata-kb đã cài trên máy bạn khác với phiên bản mà `kb-code.yml` ghim.

---

8. Giữ tri thức luôn mới

Không tài liệu nào đòi hỏi kỷ luật tài liệu hoá mới theo từng ticket từ bạn.

8.1 `-code` không cần bạn làm gì

`-code` không bao giờ được commit. `.gitignore` có dòng `.kb/*-code/`, và tài liệu được sinh lại từ working tree mỗi khi một lệnh cần đến nó: `kb svc note`, `kb build`, `kb doctor` và `kb publish` làm mới nó trước khi chạy nếu nó chưa có, cũ hơn `HEAD`, hoặc các file đang được git track có thay đổi. File mới chỉ được tính sau khi `git add`. `kb ticket check` đọc bản trên hub trước — đó là bản SA đã dùng để ground ticket (mục 4.1).

So sánh với snapshot đã publish của `kb doctor` bỏ hẳn `-code` ra ngoài (repo dev không bao giờ khớp hub ở tài liệu này); `kb publish` mới là lệnh thực sự làm mới nó và gửi lên hub.

`kb-code.yml` chạy lại `kb code-ingest` → `kb build` → `kb ci-publish` ở mỗi lần push lên nhánh mặc định, nên hub phản ánh commit hiện tại ngay khi PR của nó được merge.

Kích hoạt	Các bước	Publish
push lên nhánh mặc định, hoặc chạy tay	code-ingest → build → ci-publish	có
pull request	build (sinh lại `-code` từ merge ref)	**không bao giờ**

`-code` chỉ có một nguồn sinh là `kb code-ingest`: `kb summarize` từ chối nó. Bản trên hub từng được summarize trước 1.4.0 sẽ trở về L1/L2 deterministic ở lần CI publish kế tiếp — diff một lần, không mất thông tin; L3 không đổi.

`--scaffold-svc` cố ý không bao giờ được truyền trong CI. CI không được tạo nội dung `pending`, vì như vậy chính bước build của nó sẽ fail.

8.2 `-svc` tích luỹ ở bước handover

`dev-handover` chạy lệnh sau cho mỗi service mà ticket chạm tới:

```
kb svc note payments-api --ticket ABC-123 \
 --title "Add refund endpoint" --refs "payments-spec §4.2"
```

Lệnh đó thêm một dòng vào `hist.<service>`. Nó tất định và **idempotent**: chạy lại với cùng ticket và cùng service sẽ cập nhật dòng đó chứ không nhân đôi.

Bạn không bao giờ sửa tay `hist.*`. Đó là nhật ký chỉ thêm, chỉ do `kb svc note` ghi. Những section này là sự thật — "ticket này đã chạm tới service này" — không phải khẳng định, nên chúng không cần review và không bao giờ chặn build.

`kb svc note` thoát 1 khi gặp một trong các trường hợp: không phân giải được id repo; thiếu tài liệu `-svc` hoặc `-code` đi kèm; service không xác định (một lỗi gõ không được phép bịa ra service); manifest không đọc được; hoặc manifest và file lệch nhau.

8.3 Trôi dạt trong phần mô tả trách nhiệm

Phần mô tả trách nhiệm vẫn có thể trôi dạt, và hệ thống **tự báo chứ không tự chữa**.

Khi `kb code-ingest --scaffold-svc` làm mới bằng chứng code của một service mà phần tóm tắt của section đó đã ở trạng thái `reviewed`, nó so sánh bằng chứng cũ và mới ở dạng **một chuỗi nguyên khối đã cắt khoảng trắng hai đầu** — không phải một phép diff ngữ nghĩa trên từng trường. **Bất kỳ** thay đổi nào ngoài khoảng trắng đầu/cuối đều làm nổi cờ `stale-risk: svc.<name>` trong báo cáo.

Một thay đổi thật sẽ kích hoạt nó — thêm hoặc bớt một file, một bảng. Nhưng một thay đổi chỉ ở *cách* bằng chứng được kết xuất, dù bên dưới không có dòng code nào đổi, cũng kích hoạt. Đó không phải lỗi, và nó không bao giờ đụng vào phần tóm tắt. Đó là một dấu hiệu để con người nhìn lại.

---

9. Xử lý sự cố



Bản thân bước kiểm tra hub-lag chạy trọn một lượt `code-ingest` trong một worktree tạm ở mỗi lần `kb doctor` — hãy tính thêm thời gian này vào lệnh. Trên Windows, cùng một khác biệt case của glob theo hệ điều hành đã nêu ở §7.4 có thể khiến một repo không hề đổi gì báo thành `lags`; một kết luận lag biến mất khi chạy trên Linux (CI) chính là do lưu ý đó, không phải drift thật.

---

10. Tóm tắt lệnh

Lệnh	Mục đích	Mã thoát
`kb mcp-setup [--hub-url URL] [--no-verify]`	Ghi thông tin xác thực HTTP MCP của hub vào `.env` và xác minh chúng	`0` ok, `1` thiếu giá trị hoặc kiểm tra kết nối thất bại
`kb code-ingest [--db p] [--tags t] [--scaffold-svc] [--json]`	Trích cấu trúc code vào `-code` (`--db`/`--tags` thay thế `code_ingest:` cho lượt chạy đó)	`0` ok, `1` không phát hiện gì hoặc từ chối đích đến
`kb svc note <svc> --ticket <id> --title "…" [--no-refresh]`	Thêm một dòng vào `hist.<svc>`	`0` ok, `1` service không xác định hoặc thiếu tài liệu
`kb build [--strict] [--no-refresh]`	Kiểm tra kho	`0` ok, `1` lỗi
`kb doctor [--no-refresh]`	Kiểm tra sức khoẻ KB (repo dev: làm mới `-code` trước)	`0` ok, `1` lỗi, `2` trích dẫn cũ
`kb approve <doc> [--section <id>]`	Chuyển các section đã sửa sang `reviewed`	`0` ok
`kb publish --pr`	Mở PR trên hub (lượt publish đầu của bước seed)	`0` ok
`kb ticket fetch <id> [--out p] [--hub u]`	Lấy một ticket từ hub vào `docs/impl/<id>/ticket.md`	`0` ok, `1` không có trên hub, mơ hồ, hoặc id không an toàn
`kb ticket check <file> [--hub u]`	Grounding của ticket so với `-code` trên hub; revision bị lệch chỉ fail khi một fact được trích đã mất	`0` PASS, `1` FAIL
`kb mission next --from-hub [--repo id]`	Trạng thái mọi story đã publish, chỉ từ hub	`0`
`kb resolve <file>`	Kiểm tra trích dẫn của ticket	`0` ok, `1` hỏng, `2` cũ
`kb diff <doc> --against <rev>`	Sửa đổi đã thay đổi những gì	`0`
`kb query <text>` / `kb get <doc> <section>`	Tìm và lấy nội dung	`0`
`kb plan waves <plan-file> [--json]`	Các wave dependency của một `docs/impl/<id>/plan.md`	`0` ok, `1` file dùng chung không có dependency, vòng lặp, task không xác định, hoặc thiếu `Depends on:`
`kb impl tidy [--impl-dir]`	Dời layout phẳng cũ `docs/impl/<id>-*` vào `docs/impl/<id>/`; chạy lại không đổi gì, không bao giờ ghi đè	`0` sau khi in báo cáo, `1` khi thư mục không tồn tại
`kb pr lint <file>`	Cổng bằng chứng cho PR	`0` PASS, `1` FAIL

Slash command: `/dev-implement-ticket`, `/dev-design`, `/dev-plan`, `/dev-execute`, `/dev-handover`, `/dev-code-seed`.

---

11. Những tên được dành riêng

`<repo_id>-code` và `<repo_id>-svc` được dành riêng cho tri thức về code của chính repository này. Một tài liệu nghiệp vụ tuyệt đối không được lấy một trong hai hậu tố đó.

Tiền tố section là một hợp đồng đã công bố mà công cụ của BA và các công cụ tương lai dựa vào. Mỗi tiền tố có đúng một chủ sở hữu:

Tiền tố	Tài liệu	Chủ sở hữu
`struct.tree`, `svc.<name>`, `db.<table>`, `dep.<eco>`, `int.<name>`, `api.<tag>`, `cmd.<purpose>`	`-code`	extractor
`svc.<name>`	`-svc`	bạn (model soạn nháp)
`flow.<name>`	`-svc`	bạn
`hist.<name>`	`-svc`	chỉ `kb svc note`

`svc.<name>` trùng nhau ở cả hai tài liệu **là có chủ đích**. Đó là khoá nối cho phép một `Container(alias, label, technology, description)` trong C4 được điền từ `-code` (ba tham số đầu) và `-svc` (tham số thứ tư).